🛠️ GUÍA TÉCNICA DETALLADA

Solución 1: Escuchadores Individuales

✍️ Por Ramón Abramo • Profesor de Desarrollo Web

¡Hola de nuevo! Soy Ramón Abramo. En este documento vamos a desmenuzar línea por línea la **Solución 1**. Aquí no usamos el borboteo de eventos hacia el contenedor padre: vinculamos un `addEventListener('click')` directamente a cada botón. Es el enfoque más transparente y pedagógico para que el alumno entienda exactamente qué hace cada tecla.

🎯 1. ¿Cómo se concibe la Solución 1?

Cuando un alumno empieza con JavaScript, pedirle que entienda *Event Bubbling*, `closest()` y `dataset` de golpe puede ser contraproducente. Por eso, en la Solución 1 aplicamos una regla intuitiva:

🔢 Teclas de Números

Seleccionamos los 10 botones numéricos y los recorremos con un `forEach` asignando un click a cada uno.

➕ Teclas de Operadores

Seleccionamos los 4 operadores (+, -, *, /) y los recorremos con otro `forEach`.

🎯 Teclas Únicas

Los botones `=` y `C` tienen cada uno su propio `addEventListener` individual.

📂 Archivos de esta Carpeta (`version-1-escuchadores-individuales/`)

- `app.js` : La solución completa y comentada con cada hito.
- `app.alumno.js` : La plantilla con los bloques `TODO` vacíos para resolver en clase.
- `GUIA_PASO_A_PASO.md` : Documento de referencia rápida en Markdown.
- Las diapositivas (`presentacion_solucion_1_escuchadores.html`) están en la carpeta `documentacion/` .

1. Selección de Nodos en el DOM

> Para conectar cada botón por separado, necesitamos seleccionar las listas de nodos (`NodeList`) mediante selectores de atributos CSS:

```
// Pantalla LCD
const display = document.getElementById('display');

// Grupos de botones seleccionados con querySelectorAll (NodeList)
const botonesNumeros = document.querySelectorAll('[data-numero]');
const botonesOperadores = document.querySelectorAll('[data-operador]');

// Botones únicos con querySelector
const botonIgual = document.querySelector('[data-accion="igual"]');
const botonBorrar = document.querySelector('[data-accion="borrar"]');
```

2. Las 4 Variables de Estado en Detalle

La calculadora necesita memoria volátil. A mis alumnos siempre les dibujo este esquema en la pizarra:

Variable	Tipo y Valor	¿Por qué es indispensable?
`operandoActual`	`String: '0'`	Contiene lo que se ve en el display. Es texto para poder concatenar dígitos (ej: `'1' + '2' = '12'`).
`operandoAnterior`	`String / null`	Guarda el primer número antes de que el usuario pulse un operador matemático.
`operacion`	`String / null`	Recuerda el operador pendiente: `'+'` , `'-'` , `'*'` o `'/'` .
`reiniciarDisplay`	`Boolean: false`	Bandera booleana: si vale `true` , el siguiente dígito pulsado borrará la pantalla limpia.

3. La Función de Renderizado: `actualizarDisplay()`

Centralizamos el refresco de pantalla en una función auxiliar para aplicar el principio DRY (*Don't Repeat Yourself*):

```
function actualizarDisplay() {
    display.textContent = operandoActual;
}
```

[HITO 1] Escribir Números en Pantalla

> **Objetivo:** Que al pulsar cualquier botón del 0 al 9, el número se escriba en el display respetando las reglas de concatenación.

```
function agregarNumero(numero) {
    if (reiniciarDisplay) {
        operandoActual = numero;        // Empieza un número nuevo
        reiniciarDisplay = false;
    } else {
        // Si hay un '0' inicial lo sustituimos; si no, concatenamos
        operandoActual = (operandoActual === '0') ? numero : operandoActual + numero;
    }
    actualizarDisplay();
}

// Bucle forEach para asignar click a cada botón numérico:
botonesNumeros.forEach((boton) => {
    boton.addEventListener('click', () => {
        agregarNumero(boton.dataset.numero);
    });
});
```

> 🧪 **Prueba de Fuego del Hito 1**
> Pulsad `7`, `8`, `9`. En pantalla debe verse `789`. Si pulsáis `0` al inicio, no debe salir `"08"` sino `"8"`.

[HITO 2] Guardar Memoria y la Suma (+)

> **Objetivo:** Al pulsar `+`, transferir lo que hay en pantalla a `operandoAnterior` y preparar la calculadora para el segundo número.

```
function seleccionarOperador(nuevoOperador) {
    // [HITO 5] Si ya había operación y el usuario pulsa otro operador, calcula antes
    if (operacion !== null && !reiniciarDisplay) {
        calcular();
    }

    operandoAnterior = operandoActual; // Guarda el 1er número
    operacion = nuevoOperador;         // Guarda '+'
    reiniciarDisplay = true;           // El próximo dígito limpiará la pantalla
}

// Bucle forEach para los operadores:
botonesOperadores.forEach((boton) => {
    boton.addEventListener('click', () => {
        seleccionarOperador(boton.dataset.operador);
    });
});
```

> 🧪 **Prueba de Fuego del Hito 2**
> Escribid `15`, haced clic en `+` y luego pulsad `4`. En pantalla se ve `4`. El `15` sigue a salvo en `operandoAnterior`.

[HITO 3 y 4] La Función de Cálculo: `calcular()`

Objetivo: Al pulsar `=`, convertir los operandos de texto a números reales con `parseFloat()`, resolver la operación con `switch` y mostrar el total.

```
function calcular() {
    // Si falta algún dato imprescindible, cancelamos
    if (operacion === null || operandoAnterior === null) return;

    const anterior = parseFloat(operandoAnterior);
    const actual = parseFloat(operandoActual);
    let resultado;

    switch (operacion) {
        case '+':
            resultado = anterior + actual;
            break;
        case '-':
            resultado = anterior - actual;
            break;
        case '*':
            resultado = anterior * actual;
            break;
        case '/':
            // [HITO 7] Control de división por cero
            resultado = (actual === 0) ? 'Error' : anterior / actual;
            break;
        default:
            return;
    }

    // [HITO 7] Redondeo para evitar 0.30000000000000004
    if (typeof resultado === 'number') {
        resultado = Math.round(resultado * 100000000) / 100000000;
    }

    operandoActual = resultado.toString();
    operacion = null;
    operandoAnterior = null;
    reiniciarDisplay = true;
    actualizarDisplay();
}

// Conexión directa del botón igual (=):
botonIgual.addEventListener('click', () => calcular());
```

🧪 Pruebas de fuego de Hito 3 y 4

- **Suma:** `7 + 8 =` → **15**
- **Resta:** `9 - 4 =` → **5**
- **Multiplicación:** `6 * 3 =` → **18**
- **División:** `20 / 4 =` → **5**

[HITO 5] Operaciones Encadenadas

> **Objetivo:** Si el usuario teclea `5 + 3 + 2 =`, al pulsar el segundo `+` la calculadora debe calcular primero `8` de forma automática sin obligar a presionar `=`.

Fijaos en la condición que colocamos al principio de `seleccionarOperador()`:

```
if (operacion !== null && !reiniciarDisplay) {
    calcular(); // Resuelve 5 + 3 = 8 antes de registrar el nuevo operador
}
```

[HITO 6] Botón de Borrado Total (C)

> **Objetivo:** Al pulsar `C`, restablecer toda la calculadora a sus valores iniciales.

```
function borrar() {
    operandoActual = '0';
    operandoAnterior = null;
    operacion = null;
    reiniciarDisplay = false;
    actualizarDisplay();
}

// Conexión directa del botón borrar (C):
botonBorrar.addEventListener('click', () => borrar());
```

[HITO 7] Control de Casos Límite

División entre Cero

En JavaScript, `5 / 0` devuelve `Infinity`. Para que sea como una calculadora real, comprobamos si `actual === 0` y asignamos el string `'Error'`.

Redondeo Flotante

Al multiplicar por `100000000` y dividir, recortamos los decimales fantasma de IEEE 754 (como `0.1 + 0.2 = 0.30000000000000004`).

📊 Balance Técnico de la Solución 1

Analicemos con honestidad docente las ventajas y los límites de este enfoque:

✅ Ventajas Pedagógicas

- **Comprensión inmediata:** El alumno ve que el botón 7 dispara su propia función.
- No requiere entender el borboteo ni el objeto `evento`.
- Fácil de depurar con `console.log` en cada botón.

⚠️ Limitaciones Técnicas

- Crea **16 escuchadores en memoria** simultáneamente.
- Código repetitivo en la vinculación de eventos.
- Si creamos un botón nuevo por DOM dinámico, no responde al click.

🚀 Retos Opcionales de Ampliación para Alumnos

1. Tecla de Coma Decimal (.)

Añadir el botón `.` en HTML y modificar `agregarNumero` para que no permita más de un punto: `if (numero === '.' && operandoActual.includes('.')) return;`.

2. Soporte para Teclado Físico

Escuchar `window.addEventListener('keydown', (e) => ...)` para que el usuario pueda usar el teclado numérico del ordenador.

¡Enhorabuena! Has completado el análisis exhaustivo de la Solución 1. ▮✨